BTS CIEL · PROGRAMMATION C++ · TRAVAUX PRATIQUES

TP08 — Rapport de supervision à partir d'un journal de mesures

Prérequis : chapitres 1 à 8 · **Durée indicative :** 6 heures · **Travail :** en binôme · **TP de synthèse**

Contexte professionnel

Les capteurs d'un bâtiment envoient leurs mesures à un collecteur, qui les ajoute ligne par ligne à un **journal** texte. Le journal n'est pas parfait : certaines lignes sont tronquées, vides ou contiennent des valeurs illisibles, suite à des coupures de communication. Un second fichier, rédigé par l'exploitant, donne les **seuils** de fonctionnement normal de chaque capteur.

Chaque matin, le technicien a besoin d'un **rapport de synthèse** par capteur : nombre de mesures, moyenne, extrêmes et nombre de dépassements de seuils. Les lignes défectueuses du journal doivent être signalées, sans bloquer le traitement.

Objectifs

- Lire deux fichiers texte de formats différents, en vérifiant chaque ouverture.
- Analyser une ligne de texte et convertir une valeur en traitant les erreurs par exceptions.
- Regrouper des mesures par capteur dans un `std::vector` de `struct`.
- Produire un fichier de rapport et vérifier son écriture.
- Mobiliser l'ensemble des notions du cours dans un programme structuré.

Compétences mobilisées

Compétence	Mise en œuvre dans ce TP
C04 — Analyser un système informatique	Analyser les formats de fichiers et les défauts possibles du journal
C05 — Concevoir un système informatique	Concevoir les structures de données et le découpage en fonctions
C06 — Valider un système informatique	Valider le programme sur le journal de test et sur des fichiers erronés
C08 — Coder	Écrire la lecture, l'analyse, l'agrégation et l'écriture du rapport

Cahier des charges

Fichier `seuils.txt`

Une ligne par capteur : identifiant, seuil bas, seuil haut, séparés par des espaces.

```
T1 15 27
T2 15 27
H1 30 60
```

Si une donnée de ce fichier est illisible, le programme s'arrête avec un message d'erreur et le code de retour 1 : on ne travaille pas avec des seuils douteux.

Fichier `releves.txt` (journal)

Une mesure par ligne : identifiant, **un espace**, valeur. Le fichier de départ contient volontairement des lignes défectueuses :

```
T1 21.5
T2 29.0
H1 45.0
T1 22.5
T2 abc
H1 61.5
T3
T1 30.5

H1 50.0
T2 26.0
P1 1013.0
```

Une ligne est **invalide** si elle ne contient pas d'espace, si l'identifiant est vide, ou si le texte après le premier espace n'est pas convertible en nombre avec `std::stod`. Chaque ligne invalide est signalée sur `std::cerr` avec son numéro, puis ignorée.

Traitement

- Les mesures valides sont regroupées par capteur, dans l'ordre de première apparition.
- Pour chaque capteur : nombre de mesures, moyenne, minimum, maximum et nombre de mesures **hors seuils** (strictement inférieures au seuil bas ou supérieures au seuil haut).
- Un capteur absent de `seuils.txt` est traité normalement, mais sa colonne « Hors seuils » affiche `?`.

Fichier `rapport.txt` produit

```
Rapport de supervision
Lignes lues : 12
Lignes invalides : 3
Capteur ; Mesures ; Moyenne ; Minimum ; Maximum ; Hors seuils
T1 ; 3 ; 24.8333 ; 21.5 ; 30.5 ; 1
T2 ; 2 ; 27.5 ; 26 ; 29 ; 1
H1 ; 3 ; 52.1667 ; 45 ; 61.5 ; 1
P1 ; 1 ; 1013 ; 1013 ; 1013 ; ?
```

En fin d'exécution, la console affiche : `Rapport ecrit dans rapport.txt : 4 capteur(s), 3 depassement(s)`.

Structures et fonctions imposées

```
struct Seuils {
    std::string identifiant{};
    double minimum{};
    double maximum{};
};

struct Bilan {
    std::string identifiant{};
    int nbMesures{};
    double somme{};
    double minimum{};
    double maximum{};
    bool seuilsConnus{};
    int nbHorsSeuils{};
};
```

Fonction	Rôle
`chargerSeuils`	Lit `seuils.txt` dans un `std::vector<Seuils>` ; renvoie `false` en cas d'ouverture impossible ou de donnée illisible
`decouperLigne`	Sépare une ligne en identifiant et valeur ; renvoie `false` si la ligne est invalide
`chercherSeuils`	Renvoie l'adresse des seuils d'un capteur, ou `nullptr`
`chercherBilan`	Renvoie l'adresse du bilan d'un capteur, ou `nullptr`
`ajouterMesure`	Met à jour un bilan avec une nouvelle mesure
`ecrireRapport`	Écrit `rapport.txt` ; renvoie `false` en cas d'échec d'ouverture ou d'écriture

Travail demandé

Partie 1 — Analyse

1. Pour le journal de départ, listez les lignes invalides et leur cause, puis calculez à la main le rapport attendu.
2. Écrivez les prototypes des six fonctions en justifiant chaque passage de paramètre.
3. Décrivez en quelques lignes l'algorithme de `decouperLigne` : comment trouver le premier espace et construire les deux morceaux avec les outils vus en cours (indices, `length()`, concaténation de caractères) ?

Partie 2 — Lecture des fichiers

4. Écrivez `chargerSeuils` et `chercherSeuils`.
5. Écrivez `decouperLigne` et testez-la séparément sur : `"T1 21.5"`, `"T2 abc"`, `"T3"`, `""`, `" 21.5"`.

Partie 3 — Agrégation

6. Écrivez `chercherBilan` et `ajouterMesure`, puis la boucle principale de lecture du journal.

Partie 4 — Rapport

7. Écrivez `ecrireRapport` et le message final.

Partie 5 — Validation

8. Vérifiez `rapport.txt` pour le journal de départ.
9. Testez les cas d'erreur du tableau ci-dessous.

Contraintes techniques

- Compilation avec `g++ -std=c++17 -Wall -Wextra`, sans erreur ni avertissement.
- Chaque ouverture de fichier est vérifiée ; l'écriture du rapport est vérifiée après `close()`.
- `std::stod` est appelé dans un bloc `try` / `catch` (`std::invalid_argument`, `std::out_of_range`).
- Aucune copie inutile des vecteurs ; tout pointeur est testé avant usage.
- Après un `push_back` sur le vecteur des bilans, on n'utilise plus un pointeur obtenu **avant** ce `push_back`.

Tests à effectuer

N°	Situation	Résultat attendu	Code
1	Fichiers de départ	`rapport.txt` conforme ; lignes 5, 7 et 9 signalées sur `std::cerr`	0
2	`seuils.txt` absent (renommé)	Message d'erreur, aucun rapport	1
3	`releves.txt` absent	Message d'erreur, aucun rapport	1
4	`seuils.txt` contenant `T1 15 abc`	Message d'erreur (seuils illisibles)	1
5	`releves.txt` vide	Rapport avec 0 ligne lue et aucun capteur	0
6	Ligne `T1 12abc` ajoutée au journal	Comptée comme `12` (comportement de `std::stod`) : à commenter dans le compte rendu	0

Livrables

- Le fichier `supervision.cpp`.
- La feuille d'analyse (partie 1), le fichier `rapport.txt` obtenu et le compte rendu des tests, dont le commentaire du test 6.

Critères d'évaluation

Critère	Indicateurs de réussite	Points
Analyse et conception	Lignes invalides identifiées, rapport calculé à la main, prototypes justifiés, algorithme de découpage clair	4
Lecture des fichiers	Ouvertures vérifiées ; seuils illisibles détectés ; journal lu ligne par ligne	4
Analyse des lignes	Découpage correct, exceptions de `std::stod` traitées, lignes invalides signalées avec leur numéro	4
Agrégation	Bilans exacts par capteur, capteur sans seuils traité, pointeurs testés	4
Rapport	Format conforme, écriture vérifiée, message final	2
Tests	Six tests déroulés et commentés	2
Total		**20**